Difference Between TL;DR and ELI-15 Prompting

Try prompting using TL;DR and ELI-15 and observe the difference in output.

Example Topic: “What is Automation Testing?”

---

1. TL;DR Prompting

Prompt

“TL;DR Explain Automation Testing”

Output

Automation Testing uses tools and scripts to execute test cases automatically instead of manual testing.

---

2. ELI-15 Prompting

Prompt

“ELI-15 Explain Automation Testing”

Output

Automation Testing is like teaching a robot to test an application for you. Instead of a person clicking buttons and checking results every time, automated scripts perform the same actions repeatedly and quickly. This saves time, reduces human effort, and helps test applications faster and more accurately.

Observation

Aspect	TL;DR	ELI-15
Purpose	Quick summary	Simple understanding
Explanation Style	Short and direct	Detailed and beginner-friendly
Vocabulary	Technical	Easy language
Examples	Rarely included	Commonly included
Audience	Experienced users	New learners

Differences (TL;DR & ELI Prompting):

- TL;DR gives a concise technical explanation in one or two lines.
- ELI-15 explains the same concept in simple language with relatable examples.
- In software testing, TL;DR is useful for quick documentation or notes, while ELI-15 helps beginners understand testing concepts easily.

2. Use structured reasoning and step by step prompting to generate

Automation frame work:

Context
Act like an Automation Assistant with E-Commerce application knowledge

Instructions:

Generate a Selenium Java-based automation framework
Verify the Add to Cart functionality in an E-Commerce application
Include setup and teardown methods using TestNG
Use explicit waits instead of Thread.sleep()
Validate both successful and unsuccessful Add to Cart scenarios
Validate product quantity and cart total updates
[MANDATORY] Only TestNG
[MANDATORY] Use only BeforeMethod and AfterMethod in TestNG Annotations
[MANDATORY] Follow Page Object Model (POM) design pattern
[MANDATORY] Include reusable utility classes for waits and screenshots
[MANDATORY] Generate Extent Report integration
[MANDATORY] Use DataProvider for multiple product test data
[MANDATORY] Validate error message when product is out of stock

Tone:
[STRICTLY] Professional tone in review comments and execution summary
Output:

only code without any markdown or extra text

Persona:
Act as a Test Automation Architect to review the framework implementation and provide professional review comments on framework scalability, reusability, and maintainability

Output:

```
package com.ecommerce.base;

import com.ecommerce.utils.ExtentManager;

import com.aventstack.extentreports.ExtentReports;

import com.aventstack.extentreports.ExtentTest;

import io.github.bonigarcia.wdm.WebDriverManager;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.testng.annotations.AfterMethod;

import org.testng.annotations.BeforeMethod;

import java.time.Duration;

public class BaseTest {

protected WebDriver driver;

protected ExtentReports extent = ExtentManager.getInstance();
```

```
protected ExtentTest test;
@BeforeMethod
public void setUp() {
    WebDriverManager.chromedriver().setup();
    driver = new ChromeDriver();
    driver.manage().window().maximize();
    driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(30));
}

@AfterMethod
public void tearDown() {
    if (driver != null) {
        driver.quit();
    }
    extent.flush();
}
}
package com.ecommerce.utils;
```

```
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.io.File;
import java.io.IOException;
import java.time.Duration;
public class WaitUtils {
private WebDriver driver;
private WebDriverWait wait;
public WaitUtils(WebDriver driver) {
   this.driver = driver;
   this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
}

public WebElement waitForVisibility(By locator) {
   return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
}
```

```
public WebElement waitForClickable(By locator) {
    return wait.until(ExpectedConditions.elementToBeClickable(locator));
}

public static String captureScreenshot(WebDriver driver, String name) {
    String path = System.getProperty("user.dir") + "/reports/screenshots/" + name + ".png";
    File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
    try {
        FileUtils.copyFile(src, new File(path));
    } catch (IOException e) {
        e.printStackTrace();
    }
    return path;
}
}
package com.ecommerce.utils;
import com.aventstack.extentreports.ExtentReports;
```

```
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
public class ExtentManager {
private static ExtentReports extent;
public static ExtentReports getInstance() {
    if (extent == null) {
        ExtentSparkReporter spark = new ExtentSparkReporter("reports/ExtentReport.html");
        extent = new ExtentReports();
        extent.attachReporter(spark);
    }
    return extent;
}
}
package com.ecommerce.pages;
import com.ecommerce.utils.WaitUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class ProductPage {
private WebDriver driver;
```

```
private WaitUtils waitUtils;
private By searchBox = By.id("search");
private By searchBtn = By.cssSelector(".search-button");
private By addToCartBtn = By.id("add-to-cart");
private By outOfStockMsg = By.cssSelector(".stock-error");
private By cartIcon = By.id("cart-btn");

public ProductPage(WebDriver driver) {
   this.driver = driver;
   this.waitUtils = new WaitUtils(driver);
}

public void searchProduct(String productName) {
   waitUtils.waitForVisibility(searchBox).sendKeys(productName);
   waitUtils.waitForClickable(searchBtn).click();
}

public void addToCart() {
```

```
    waitUtils.waitForClickable(addToCartBtn).click();
}

public String getErrorMessage() {
    return waitUtils.waitForVisibility(outOfStockMsg).getText();
}

public void goToCart() {
    waitUtils.waitForClickable(cartIcon).click();
}
}
package com.ecommerce.pages;
import com.ecommerce.utils.WaitUtils;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class CartPage {
private WebDriver driver;
private WaitUtils waitUtils;
```

```
private By cartTotal = By.id("total-price");
private By productQty = By.className("item-qty");

public CartPage(WebDriver driver) {
    this.driver = driver;
    this.waitUtils = new WaitUtils(driver);
}

public String getTotal() {
    return waitUtils.waitForVisibility(cartTotal).getText();
}

public String getQuantity() {
    return waitUtils.waitForVisibility(productQty).getAttribute("value");
}
}
package com.ecommerce.tests;
import com.ecommerce.base.BaseTest;
```

```
import com.ecommerce.pages.CartPage;
import com.ecommerce.pages.ProductPage;
import com.ecommerce.utils.WaitUtils;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
public class CartTest extends BaseTest {
@DataProvider(name = "productData")
public Object[][] getProductData() {
    return new Object[][] {
        {"Laptop", true},
        {"OutofStockPhone", false}
    };
}

@Test(dataProvider = "productData")
public void validateAddToCart(String product, boolean isAvailable) {
    test = extent.createTest("Add to Cart Validation: " + product);
```

```
ProductPage productPage = new ProductPage(driver);
CartPage cartPage = new CartPage(driver);

driver.get("[https://ecommerce-app-example.com](https://ecommerce-app-example.com)");
productPage.searchProduct(product);

if (isAvailable) {
    productPage.addToCart();
    productPage.goToCart();

    Assert.assertEquals(cartPage.getQuantity(), "1", "Quantity mismatch");
    Assert.assertNotEquals(cartPage.getTotal(), "$0.00", "Total not updated");
    test.pass("Successfully added " + product + " to cart.");
} else {
    productPage.addToCart();
    String error = productPage.getErrorMessage();
    Assert.assertTrue(error.contains("Out of Stock"), "Error message not displayed");
    WaitUtils.captureScreenshot(driver, "OutOfStock_" + product);
```

```
        test.pass("Handled out of stock scenario for " + product);
    }
}
}
```

ARCHITECT REVIEW COMMENTS:

SCALABILITY: The implementation of the Page Object Model (POM) facilitates the seamless addition of new pages and components without impacting existing test logic.

REUSABILITY: Encapsulating waits within WaitUtils ensures that synchronization logic is centralized, reducing code duplication and enhancing reliability across the suite.

MAINTAINABILITY: The use of TestNG DataProviders separates test data from execution logic, allowing for easy updates to test permutations through external data sources or local arrays.

ERROR HANDLING: The integration of dynamic screenshots and Extent Reports provides high observability into test failures, which is critical for CI/CD pipeline stability.

RECOMMENDATION: To further enhance the framework, consider implementing a configuration reader for environment-specific variables and a BasePage to initialize common elements.

b) Test Strategy Document

CONTEXT: Act as an experienced QA Manager.

Instructions:

- Design a comprehensive Test Strategy Document for migration of a website from AEM to Sitecore.
- Ask exactly 6 relevant multiple-choice questions to gather project requirements.
- Questions must be asked one at a time.
- **[STRICT]** After receiving an answer for the current question, proceed to the next question only.
- **[MANDATORY]** Use the answers provided to customize the final Test Strategy Document.
- **[CRITICAL]** After all 6 questions are answered, generate the final Test Strategy Document in .docx format.
- **[MANDATORY]** Include testing scope, migration validation approach, regression strategy, SEO validation, integration testing, automation approach, environments, risks, mitigation plan, entry/exit criteria, and reporting metrics.
- **[STRICT]** Maintain a professional QA documentation style throughout the interaction and final output.

Question 1 of 6:

1**. What is the primary type of website being migrated from AEM to Sitecore?**

A) E-commerce Website
B) Corporate/Marketing Website
C) Healthcare/Patient Portal
D) Media/News Website
E) Banking/Financial Website
F) Other (please specify)

Answer: D

Question 2 of 6:

2. What is the expected migration approach for the Sitecore implementation?

A) Big Bang Migration (entire website migrated at once)
B) Phased Migration (module/page-wise migration)
C) Parallel Run (AEM and Sitecore run together temporarily)
D) Content-Only Migration
E) Replatforming with UI Redesign
F) Not finalized yet

Answer: C and D

Question 3 of 6:

3. Which testing areas are considered most critical for this migration project?

A) Content Validation & Formatting
B) SEO & Redirect Validation
C) Third-Party Integrations/APIs
D) Performance & Load Testing
E) Accessibility & Compliance
F) Cross-browser & Mobile Responsiveness
G) All of the above

Answer: G

Question 4 of 6:

4. What level of automation coverage is expected for the migration testing?

A) Smoke Tests Only
B) Regression Suite Automation

C) End-to-End Critical User Journeys
D) API + UI Automation
E) Full Automation Coverage wherever feasible
F) Manual Testing Only

Answer: F

Question 5 of 6:

5. Which environments will be available for testing during the migration?

A) QA only
B) QA + UAT
C) QA + SIT + UAT
D) QA + Staging + Production Validation
E) Full environment setup (DEV, QA, SIT, UAT, Pre-Prod)
F) Not confirmed yet

Answer: D

Question 6 of 6:

6. What is the highest business risk associated with this migration?

A) Content/Data Loss
B) SEO Ranking Impact
C) Broken Links & Navigation Issues
D) Integration Failures
E) Performance Degradation

F) Production Downtime
G) All are critical equally

Test Strategy Document:

Download the Test Strategy Document